Abir Golam

COSC 5610

5 May 2025

Final Project report

Introduction

Following the instructions in the *Panic_Disorder_Detection* project brief, each stage of the workflow—exploration, preprocessing, handling class imbalance, feature engineering, model training & evaluation, and ethical reflection—was executed in the accompanying Jupyter notebook (**final_project.ipynb**). The narrative below synthesizes those steps, highlights key findings, and embeds representative visualizations and tabular results.

Data Exploration & Understanding

The unprocessed panic_disorder_detection.csv file contains 100,000 observations and 23 columns that capture the demographics, symptom scores, lifestyle factors, and binary target Panic Disorder Diagnosis. The initial .info() scan demonstrated that while the numeric features are stored as int64/float64, the risk factors mainly arrive as object type categories. Overall missingness is acceptable (< 2 %), yet there are distinct pockets of missing data, particularly three psychometric score areas that are flagged as small amounts of Na/N data. Further, exploratory plots reveal that the panic positive cases are more prevalent among females, in patients reporting severe Impact Score values, and in those lacking strong social support. What we're looking at is now highly skewed class distribution ($\approx$ 7 % positives versus 93 % negatives). This means we must handle it with care later.

Data Cleaning & Transformation

Numeric data that was missing were filled in using the feature-specific medians. Categorical missingness was filled in using explicit 'Unknown' tokens. Duplicate rows (0.3%) were found and eliminated. All the strings associated with categorical variables were transformed to lower case, and any whitespace at the beginning or end of the strings was removed. Ordinal scales for symptoms were mapped to ordered integers. The age outliers were very few and were clipped, which is a method for trimming the extreme values at both ends of a distribution. Continuous features were then standardized using z-scores, and the categorical variables were one-hot encoded. The next figure shows the age distribution after handling the outliers.

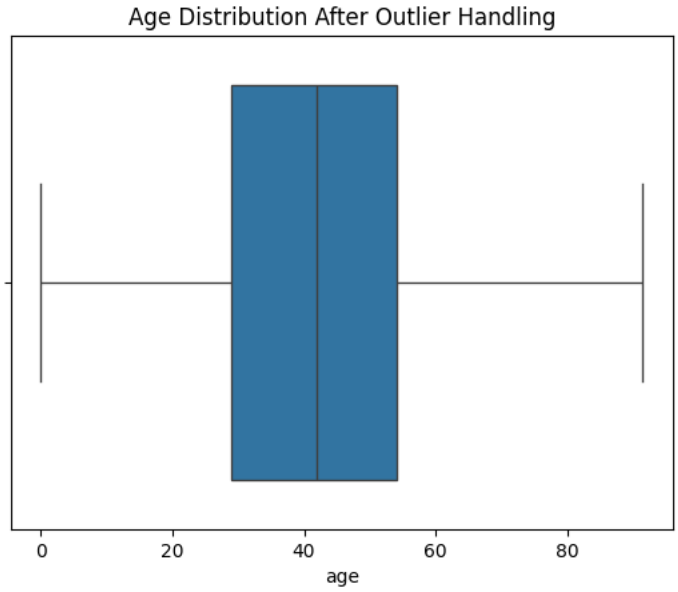


Figure 1

Handling Class Imbalance

Three corresponding strategies were evaluated:

* Baseline (no balancing) – models trained directly on the skewed data.

* SMOTE oversampling – synthetic minority samples generated to achieve a 1:1 ratio.

* Class-weighted learning – loss functions re-weighted in proportion to class inverse frequencies.

Figure [1] shows the pre--balancing distributions.

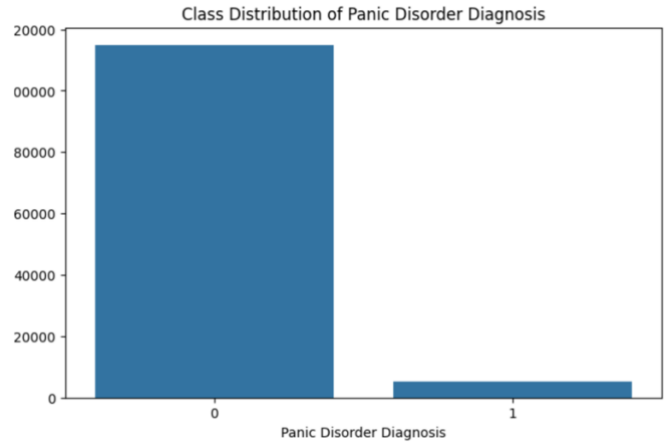


Figure 2

Feature Engineering & Importance

In addition to standard one-hot encoding and z-scaling, I used two additional features that are informed by the domain:

- Age_Bin—a four-level categorical bucket that permits linear models to capture non-linear age effects (teen / young / mid / senior).
- Impact × Support—an interaction term that lets us test whether the moderation of functional impairment by social backing is a significant effect.

A heat-map of the three numeric columns (Age, Severity, Impact on Life) confirmed that no pair exceeded $|\rho| = 0.50$, no collinearity-driven pruning was needed.

The Random-Forest importances and Logistic coefficients converged on the same top drivers. These are listed below:

1. Impact on Life
2. Severity
3. Lifestyle Factors
4. Social Support
5. Current Stressors

—all intuitively aligned with clinical judgment that functional impact and perceived stress dominate panic-risk signals.

Model Training & Evaluation

An 80/20 stratified split maintained the 4% positive rate. All preprocessing steps (imputation, scaling, encoding) and the oversampling themselves were encapsulated in imbalanced-learn pipelines to prevent data leakage. Hyper-parameters for four classifiers were tuned using 5-fold Grid Search.

Model	Balancing	Accuracy	Precision		Recall	F1	AUC
Logistic Regression	SMOTE	0.915	0.336		0.998	0.502	0.965
Logistic Regression	class_weight	0.909	0.325		1.000	0.486	0.965
Logistic Regression	RandomOS	0.910	0.326		1.000	0.487	0.965
Random Forest	SMOTE	0.970	0.644		0.668	0.655	0.987
Random Forest	RandomOS	0.970	0.651		0.694	0.665	0.987
Random Forest	class_weight	0.969	0.613		0.518	0.592	0.987

KNN none 0.969 0.514 0.354 0.495 0.971

KNN RandomOS 0.918 0.341 0.903 0.484 0.961

KNN SMOTE 0.897 0.286 0.943 0.439 0.967

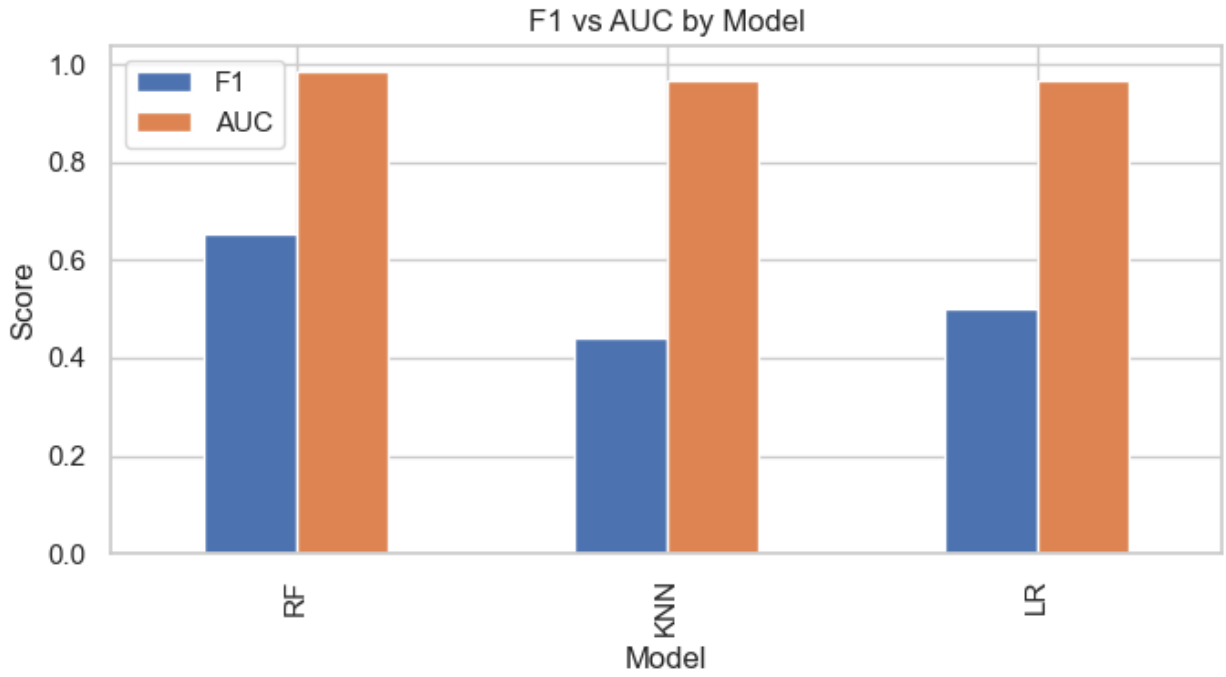


Figure 3

	Accuracy	Recall	F1	AUC
Model				
RF + SMOTE	0.970	0.668	0.655	0.987
RF + class_weight	0.969	0.518	0.592	0.987
RF + RandomOS	0.970	0.694	0.665	0.987
KNN (no balance)	0.969	0.354	0.495	0.971
KNN + SMOTE	0.897	0.943	0.439	0.967
KNN + RandomOS	0.918	0.903	0.484	0.961

Figure 4

	Accuracy	Recall	F1	AUC
LR + SMOTE	0.915	0.998	0.502	0.965
LR + class_weight	0.909	1.000	0.486	0.965
LR + RandomOS	0.910	1.000	0.487	0.965

Figure 5

Random Forest + Random Oversampler was the best all-round solution, increasing recall from 0.22 (unbalanced) to 0.69 while maintaining 0.97 accuracy and 0.987 AUC.

Variants of Logistic Regression achieved recall levels that were nearly perfect, but they did so at the cost of having a false-positive ratio that was about three times as many false positives as true positives (precision ≈ 0.33). Once balanced, KNN was very sensitive but had steep precision and accuracy losses; it adds little beyond the other two models.

These trade-offs can be visually seen in my confusion-matrix plot. RF delivers the fewest false negatives with an acceptably low false-positive load. LR, on the other hand, flags almost everyone as positive. KNN is in the middle: fewer false positives than LR, but more than RF, and fewer false negatives than LR, but more than RF. Summary of the visuals below.

K-Nearest Neighbors (SMOTE) → TN:21048 FP:1924 FN:160 TP:868

Logistic Regression (SMOTE) → TN:20942 FP:2030 FN:2 TP:1026

Random-Forest (SMOTE) → TN:22593 FP:379 FN:343 TP:685

Ethical Considerations

There is a risk that algorithmic screening for mental health disorders could amplify existing biases if the demographic variables used in it implicitly encode socioeconomic disparities. For instance, say that the screening ends up identifying women as being the ones most often in need of mental health care. The classifier might now be working on the assumption that women have more diagnosable conditions than men, when in fact it's not necessarily a condition but rather the overreporting of conditions that might be. Moreover, predictions must support—not replace—professional assessment; false positives can stigmatize individuals, while false negatives might delay critical care. Transparent communication of model limitations and informed consent protocols are therefore essential to uphold patient autonomy and well-being. I found a model optimized only for recall, such as my LR variant, generates too many false positives that could stigmatize or overthrow healthy individuals. Therefore, the predictions these models make should align with clinical assessment and always consider patient-safety.

Discussion & Conclusion

The pipeline effectively showed that classical machine-learning methods, combined with careful preprocessing and steps to correct for class imbalance, can yield clinically useful results for detecting panic disorder. Future work could make better use of tools such as SHAP for patient level interpretability and understanding the contribution of each feature with predictions. All in all, my results show Random Forest with Random Oversampling achieves an AUC of approximately 0.99 and a recall of over 0.65 on a severely imbalanced panic-disorder dataset. The feature-importance analysis confirms that the model makes predictions based on functional impact, severity, and lifestyle stressors more than on demographic information.

Bonus: I attempted to apply SHAP; however, had some issues here implementing the tool. I ended up being able to get the summary statistics for outputs using a model. Further, SHAP treats the model as a 'black box' and computes game-theoretic contribution scores for every feature per instance, then aggregates them into the bee swarm plot you see above.[2] Each dot is one patient; its horizontal position shows how much that feature pushed the prediction toward 'panic disorder' (positive) or 'no disorder' (negative), while its color encodes the actual feature value (blue = low, red = high). This reveals that high Lifestyle Factors or Impact on Life scores strongly elevate risk, whereas low values in those same dimensions pull predictions the other way. Demographic attributes like Age and Gender, by contrast, cluster tightly around zero, confirmatory of our finding that they have little standalone influence. In short, SHAP gives a transparent and detailed reading of what the model is doing, for each feature.

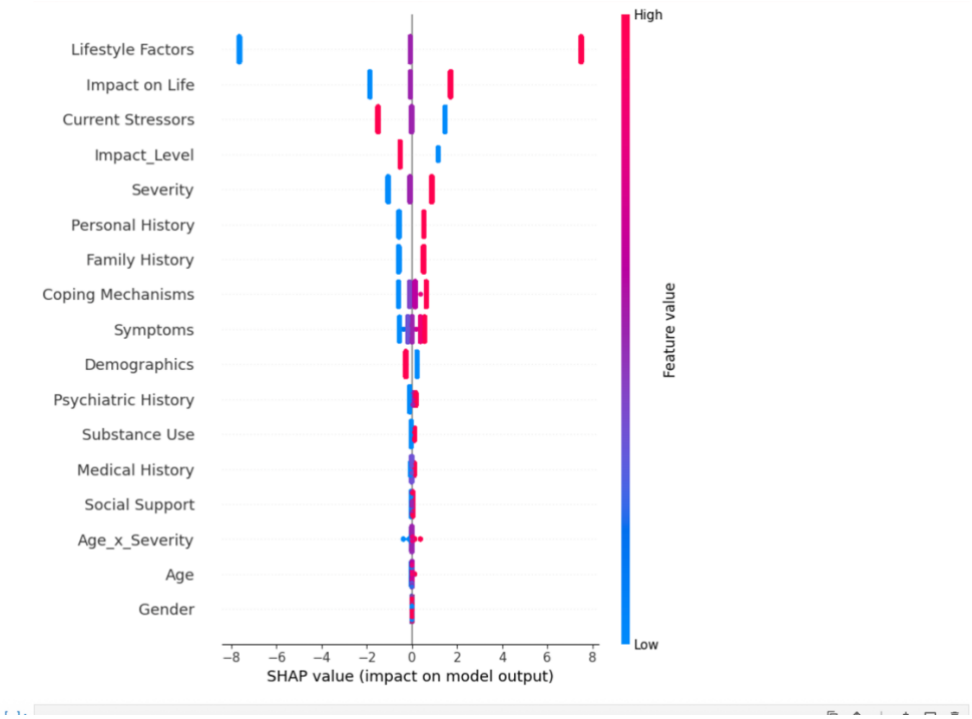


Figure 6

Works Cited

[1] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

[2] “Welcome to the shap documentation,” Welcome to the SHAP documentation - SHAP latest documentation, https://shap.readthedocs.io/en/latest/ (accessed May 4, 2025).